

Path planning of 6-DOF free-floating space robotic manipulators using reinforcement learning

Ahmad Al Ali^a, Jian-Feng Shi^b, Zheng H. Zhu^{a,*}

^a Department of Mechanical Engineering, York University, 4700 Keele Street, Toronto, M3J 1P3, Canada

^b MDA, 9445 Airport Rd, Brampton, Ontario, L6S 4J3, Canada

ARTICLE INFO

Keywords:

Space robotic manipulator
Free-floating
Path planning
Autonomous
Reinforcement learning
Deep deterministic policy gradient
Actor-critic networks
Quaternion orientation control
Collision avoidance

ABSTRACT

This paper presents a study on path planning for 6-DOF free-floating space robotic manipulators using Deep Deterministic Policy Gradient-based Reinforcement Learning. The focus is the development of a novel reward function tailored to address critical requirements for efficient and effective manipulation in space. These requirements include accurate pose alignment between the end-effector and the target, collision avoidance with both the target and other links of the manipulator, smoothing of joint velocities, adaptability to strong dynamic coupling between the manipulator and its base spacecraft due to high manipulator-spacecraft mass ratio, and resilience to noise in the state observations. Uniquely, the proposed reward function employs quaternions for orientation control to reduce pose misalignments and dynamic singularities, as opposed to traditional Euler angles. Our findings demonstrate that the Reinforcement Learning algorithm, when guided by this new reward function that integrates these enhancements and constraints, not only achieves the desired path planning objectives more efficiently but also exhibits faster convergence. Furthermore, the Reinforcement Learning successfully manages significant dynamic coupling effects caused by a high mass ratio between the robotic manipulator and the base spacecraft. Even under the challenge of noisy state observations, the trained agent successfully completes the path planning task, proving the Reinforcement Learning's applicability to real-space mission designs where the noise in observation is inevitable. The study highlights the critical role of reward function design in the Reinforcement Learning training process and its consequential impact on the solution quality, providing a solid foundation for future advancements in free-floating space robotic missions.

1. Introduction

Space robotic manipulators are increasingly used in tasks like on-orbit servicing, space debris removal, and in-space assembly of large space structures due to their high technology readiness level [1,2]. These systems typically comprise of a base spacecraft with one or multiple robotic manipulators, such as Canadarm [3]. They can be categorized into free-flying and free-floating [4]. Free-flying manipulators are attached to spacecraft with actively controlled position and orientation or pose, so they behave like fixed-base manipulators. Although extensively studied [5,6], their main drawback is high fuel consumption, which shortens their effective service life in orbit. Conversely, free-floating manipulators have a passive base spacecraft, with only the manipulator's joints being actively controlled [7,8]. This design reduces collision risks from excessive attitude control on the base spacecraft and reduces fuel consumption for longer service life. However, free-floating

manipulators experience dynamic coupling between the based spacecraft and the manipulator due to the conservation of momentum. Actions like moving the end-effector can inadvertently alter the spacecraft pose and vice versa. Consequently, the pose of the end-effector depends on the current joint angles and the previous velocities of the joints and spacecraft [9]. Due to the nonholonomic characteristics of angular momentum, path planning methods for fixed-base manipulators do not apply directly to free-floating robotic manipulators [10,11]. Thus, innovative motion control methods are needed to overcome these challenges.

2. Related works

Path planning for free-floating robotic manipulators presents unique challenges distinct from those encountered with fixed-base manipulators [12,13]. Traditional methods like inverse kinematics, while

* Corresponding author.

E-mail addresses: aiz123@yorku.ca (A. Al Ali), gzh@yorku.ca (Z.H. Zhu).

<https://doi.org/10.1016/j.actaastro.2024.08.015>

Received 12 March 2024; Received in revised form 10 August 2024; Accepted 12 August 2024

Available online 19 August 2024

0094-5765/© 2024 The Authors. Published by Elsevier Ltd on behalf of IAA. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

Table 1

Summary of path planning using RL in the literature.

Ref	Algorithm	DOF	Reward Function Expressions	Constraints Included			Noise
				EE Position	EE Attitude	Joint Velocity	
[21]	MRDDPG	2	Depends on constraints (safety, disturbance, path)	Yes	No	No	No
[21]	Soft Q-Learning	3	$R = - (t/T) \left[K_1 d^2 + K_2 \log(d^2 + \delta) \right] + K_3 \ \tau\ ^2$	Yes	No	No	No
[24]	DDPG + pre-training	3	$R = - 0.3^{0.99t} \left[K_1 d^2 + K_2 \log(d^2 + \delta) \right] + K_3 \ \tau\ ^2$	Yes	No	No	No
[25]	DDPG	3×2	$R = - (d_1 + d_2)^2 - \log(d_1 + 10^{-8}) - \log(d_2 + 10^{-8}) - v_{1,e} /(d_1^2 + 1) - v_{2,e} /(d_2^2 + 1) - 1/(100d_{\min}^2 + 0.5)$	Yes	No	Yes	No
[26]	Deep RL	6	$R_1 = (2 - d)^2 + (1 - d/2)(\sqrt{3}\pi - \theta) - 33.6$ $R_2 = \begin{cases} 2d - 0.1\theta - 1.9 & d \geq 0.1 \\ 1 + e^\theta - 10d^2 & d < 0.1 \end{cases}$ $R_3 = \begin{cases} -2d - 0.1\theta - 1.9 & d \geq 0.1 \\ -\cos(\theta) - 10d^2 & d < 0.1 \end{cases}$ $d = \sqrt{x^2 + y^2 + z^2}, \theta = \sqrt{\alpha^2 + \beta^2 + \gamma^2}$	Yes	Yes (Euler angle)	No	No
[28]	DDPG	7	$U(d, \theta) = -K_1 d + K_2 / [(d+1)(\theta+1)]$ $R = U_{t+1}(d, \theta) - U_t(d, \theta)$	Yes	Yes (Euler angle)	No	No

MRDDPG - Multi-constrained Reward Function DDPG, R is the reward gained at each time step t , t/T is a scaler that amplifies the reward in the final phase of the capture approach, d is the distance between the robot's EE (end-effector) and the target, τ is the sum of all joint torques, (δ, K_1, K_2, K_3) are the weights, $v_{1,e}, v_{2,e}$ are the EE's linear velocities for arms 1 and 2 respectively, θ is the orientation difference between the EE and the target in Euler angles (α, β, γ) , $U(d, \theta)$ and $U_{t+1}(d, \theta)$ are the potential energy at the time t and $t+1$.

straightforward and satisfying angular momentum constraints [12], often fall short of guaranteeing optimal path due to the unpredictable emergence of dynamic singularities, which can compromise manipulator performance [14]. Alternatives such as particle swarm optimization [15] and singular-free terminal sliding mode control [16] offer solutions through optimization at the cost of intensive computational loads, making them less effective in real-time applications under uncertain disturbances. To overcome these challenges, recent advancements have pivoted to machine learning [17], particularly reinforcement learning (RL) [18]. RL enables an agent to learn a decision-making policy for complex tasks in dynamic and uncertain environments, such as the robotic manipulator in this context, to maximize a pre-defined reward function [19,20]. In such a process, the agent sends commands to change the manipulator's state and receives feedback in the form of rewards based on the outcomes of that action in a Markov Decision Process (MDP) without any prior samples and rules. The training under RL is conducted in joint space, which bypasses the kinematic and dynamic singularities prevalent in the Cartesian space of free-floating robotic manipulator. If a singularity occurs and the target is not reached in the training, the RL algorithm simply discards the episode due to lack of reward, which is contingent on successful path planning.

Various RL schemes, such as Q-learning, Deep Deterministic Policy Gradient (DDPG), and Actor-Critic algorithms, have demonstrated success in addressing path planning challenges for free-floating robotic manipulators in continuous action space [21–28]. For instance, the soft Q-learning algorithm has been effectively used with a 3-DOF space robotic manipulator to guide the end-effector towards a target while minimizing joint torques [21]. Although Q-learning is suited to environments with finite action sets, policy-gradient-based methods like DDPG are better for continuous state and action spaces [22]. DDPG, in particular, has been widely applied to various manipulators. It was used for a 2-DOF space robotic manipulator in Ref. [23] with a reward function that accounted for obstacle avoidance, coupled manipulator-spacecraft dynamic disturbances, and path length constraints. Another application [24] involved the path planning for a 3-DOF space robotic manipulator, where DDPG facilitated faster convergence by incorporating pre-trainings to improve learning efficiency. Similarly, for a dual 3-DOF arm space robotic manipulator [25], DDPG was used with velocity constraints to avoid arm self-collision in its

reward function. The Actor-Critic algorithm was also applied to a 6-DOF space robotic manipulator for target capture tasks [26]. Moreover, the DDPG was used in collision-free path planning for a 7-DOF fixed-base robotic manipulator with obstacles represented by artificial potential fields [27] in the reward function. Later, such reward function was also applied to control the 7-DOF free-floating manipulator with the DDPG in [28]. However, these studies often overlook factors such as observation noise and the suppression of robotic link oscillations. Additionally, they typically presume relatively small manipulator-base spacecraft mass ratios. This assumption suggests a similarity to fixed-base robotic manipulators, which may not accurately represent the operational dynamics of certain free-floating robotic manipulator.

Table 1 summarizes the application of RL in the path planning of free-floating robotic manipulators and associated reward functions in the literature. It shows the predominant application of the DDPG algorithm across successful implementations. Most studies investigated 3-DOF robotic manipulators, with relatively fewer works for 6-DOF or beyond. A common oversight in these studies is the disregard for joint velocity constraints. Notably absent is consideration for quaternion-based attitude control and the impact of state measurement noise that is critical to real-world applications. Furthermore, the utilization of scalar reward functions in the MDP may inadvertently restrict the search space or fall into local optima, potentially due to overfitting. The challenge of striking a balance between exploration and exploitation in RL continues to be a focal point of ongoing research [29].

This paper introduces a novel reward function design in RL, implemented through the DDPG algorithm with Actor-Critic networks, for path planning of a 6-DOF free-floating robotic manipulator. This approach emphasizes the formulation of reward functions that facilitate rapid convergence in environments subject to disturbances. Key considerations include ensuring precise pose alignment of the end-effector with the target, smoothing of joint velocity, collision avoidance, mitigation of dynamic coupling between the manipulator and the base spacecraft - especially at high manipulator-base spacecraft mass ratio, and resilience to state measurement noise. The algorithm's efficacy and its advantages are demonstrated through numerical simulations.

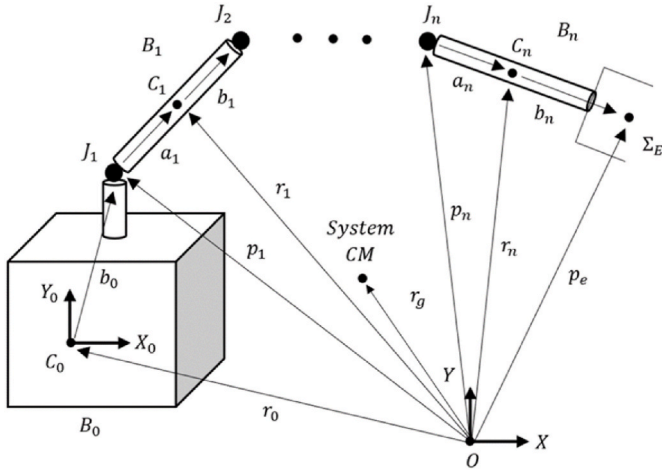


Fig. 1. Schematic of a free-floating robotic manipulator.

3. Problem statement

3.1. Free-floating robotic manipulator

RL algorithm, whether model-free or model-based, trains agents via interaction with an environment, which is a free-floating robotic manipulator in this paper. Since training with actual space robotics is impractical or cost prohibitive, a dynamic model of a free-floating space robotic manipulator must be used for training. This model, illustrated in Fig. 1, comprises of a base spacecraft and a manipulator with n rigid links connected in series by n independently actuated revolute joints. Introduce an inertial frame, OXYZ, in the free space. The base spacecraft (B_0) is assumed as link 0. The location of its center of mass (CoM), denoted as joint 0 (C_0), is defined as $\mathbf{r}_0 \in \mathbb{R}^3$. The spacecraft's rotation relative to the inertial frame is defined by the matrix $\mathbf{A}_0 \in \mathbb{R}^{3 \times 3}$. The manipulator's 1st link is connected to the base spacecraft at joint J_1 , which is denoted as $\mathbf{b}_0 \in \mathbb{R}^3$ from C_0 with a rotation matrix $\mathbf{A}_1 \in \mathbb{R}^{3 \times 3}$. Each link, labeled B_i , is connected by a joint J_i ($1 \leq i \leq n$). The position and rotation of J_i is represented by a position vector $\mathbf{p}_i \in \mathbb{R}^3$ and a unit vector $\mathbf{j}_i \in \mathbb{R}^3$, respectively. The end-effector is treated as the $(i+1)^{\text{th}}$ link. Its position and rotation are denoted by $\mathbf{p}_e = \mathbf{p}_{i+1}$ and $\mathbf{A}_e$, respectively.

In this configuration, the position ($\mathbf{p}_e$) and orientation ($\mathbf{A}_e$) of the end-effector can be determined by propagating the transformations from the base spacecraft to the end-effector, such that,

$$\mathbf{p}_e = \mathbf{r}_0 + \mathbf{b}_0 + \sum_{i=1}^n (\mathbf{p}_{i+1} - \mathbf{p}_i) \quad (1)$$

$$\mathbf{A}_e = \mathbf{A}_0 \prod_{i=1}^n \mathbf{A}_i \quad (2)$$

where $\mathbf{A}_i$ is the rotation matrix of the i th link.

Differentiating Eqs. (1) and (2) with respect to time yields the linear ($\mathbf{v}_e \in \mathbb{R}^3$) and angular ($\boldsymbol{\omega}_e \in \mathbb{R}^3$) velocities of the end-effector as

$$\mathbf{v}_e = \mathbf{v}_0 + \boldsymbol{\omega}_0 \times (\mathbf{p}_e - \mathbf{r}_0) + \sum_{i=1}^n [(\dot{\varphi}_i \mathbf{j}_i) \times (\mathbf{p}_e - \mathbf{p}_i)] \quad (3)$$

$$\boldsymbol{\omega}_e = \boldsymbol{\omega}_0 + \sum_{i=1}^n \boldsymbol{\omega}_i, \boldsymbol{\omega}_i = \dot{\varphi}_i \mathbf{j}_i \quad (4)$$

where $\mathbf{v}_0 \in \mathbb{R}^3$ and $\boldsymbol{\omega}_0 \in \mathbb{R}^3$ are the linear and angular velocities of the spacecraft, respectively, $\boldsymbol{\omega}_i = \dot{\varphi}_i \mathbf{j}_i$ is the i th joint angular velocity with $\mathbf{j}_i \in \mathbb{R}^3$ being its direction in the inertial frame.

Define the states of the base spacecraft and the end-effector as $\mathbf{V}_b = [\mathbf{v}_0, \boldsymbol{\omega}_0]^T$ and $\mathbf{V}_e = [\mathbf{v}_e, \boldsymbol{\omega}_e]^T$, respectively. Equations (3) and (4) can be rewritten in a compact form

$$\begin{cases} \mathbf{V}_e = \mathbf{J}_b \mathbf{V}_b + \mathbf{J}_m \dot{\boldsymbol{\Phi}} \\ \mathbf{J}_b = \begin{bmatrix} \mathbf{I} & \langle \mathbf{r}_0 - \mathbf{p}_e \rangle \\ 0 & \mathbf{I} \end{bmatrix} \in \mathbb{R}^{6 \times 6} \\ \mathbf{J}_m = \begin{bmatrix} \langle \mathbf{p}_1 - \mathbf{p}_e \rangle \mathbf{j}_1 & \langle \mathbf{p}_2 - \mathbf{p}_e \rangle \mathbf{j}_2 & \cdots & \langle \mathbf{p}_n - \mathbf{p}_e \rangle \mathbf{j}_n \\ \mathbf{j}_1 & \mathbf{j}_2 & \cdots & \mathbf{j}_n \end{bmatrix} \in \mathbb{R}^{6 \times n} \end{cases} \quad (5)$$

where $\dot{\boldsymbol{\Phi}} = [\dot{\varphi}_1, \dot{\varphi}_2, \dots, \dot{\varphi}_n]^T$ is the vector of joint angular velocity, and ($\mathbf{J}_b$, $\mathbf{J}_m$) are the Jacobian matrices dependent on the base spacecraft and manipulator motions. Here, $\langle \cdot \rangle$ denotes a 3×3 skew-symmetric matrix of corresponding vectors of the cross product. The detailed expressions of $\mathbf{J}_b$ and $\mathbf{J}_m$ are given in Ref. [30].

The assumption of a free-floating space robotic manipulator implies that the conservation of linear and angular momentums of the system in free space because no thrust is applied to the base spacecraft. For zero initial momentums, the conservation can be expressed as

$$\begin{bmatrix} \mathbf{P} \\ \mathbf{L} \end{bmatrix} = \mathbf{M}_b \mathbf{V}_b + \mathbf{M}_m \dot{\boldsymbol{\Phi}} = 0 \quad (6)$$

with

$$\mathbf{P} = m_0 \mathbf{v}_0 + \sum_{i=1}^n m_i (a_i \dot{\mathbf{p}}_i + b_i \dot{\mathbf{p}}_{i+1}), \mathbf{L} = (\mathbf{I}_0 \boldsymbol{\omega}_0 + \mathbf{r}_0 \times m_0 \mathbf{v}_0) + \sum_{i=1}^n [\mathbf{I}_i \boldsymbol{\omega}_i + \mathbf{r}_i \times m_i (a_i \dot{\mathbf{p}}_i + b_i \dot{\mathbf{p}}_{i+1})]$$

where m_i ($i \in [0, n]$) and $\mathbf{I}_i \in \mathbb{R}^{3 \times 3}$ are the i th link mass and inertia moment at its CoM, (a_i, b_i) are the distances from the CoM to the two ends, $\mathbf{M}_b \in \mathbb{R}^{6 \times 6}$ is the inertia matrix of the base spacecraft, and $\mathbf{M}_m \in \mathbb{R}^{n \times n}$ is the coupling inertia matrix of the manipulator. The detailed expression of these inertia matrices can be found in Ref. [30].

Substituting Eq. (5) into (6) yields

$$\mathbf{V}_e = (\mathbf{J}_m - \mathbf{J}_b \mathbf{M}_b^{-1} \mathbf{M}_m) \dot{\boldsymbol{\Phi}} = \mathbf{J}_g \dot{\boldsymbol{\Phi}} \quad (7)$$

where $\mathbf{J}_g \in \mathbb{R}^{6 \times n}$ is the generalized Jacobian matrix dependent on the base spacecraft attitude, joint angles, and mass properties of the base spacecraft and the links.

Accordingly, the dynamics of the free-floating robotic manipulator can be derived as

$$\mathbf{M} \ddot{\mathbf{q}} + \dot{\mathbf{M}} \dot{\mathbf{q}} + \mathbf{C}(\dot{\mathbf{q}}, \mathbf{q}) = \mathbf{F} \quad (8)$$

where

$$\mathbf{M} = \begin{bmatrix} \mathbf{M}_b & \mathbf{M}_{bm} \\ \mathbf{M}_{bm}^T & \mathbf{M}_m \end{bmatrix}, \mathbf{F} = \begin{bmatrix} \mathbf{0} \\ \mathbf{u} \end{bmatrix}, \mathbf{C}(\dot{\mathbf{q}}, \mathbf{q}) = \begin{bmatrix} \mathbf{c}_b \\ \mathbf{c}_m \end{bmatrix}$$

$$\mathbf{c}_b = -\frac{1}{2} \frac{\partial}{\partial \mathbf{x}_b} (\mathbf{V}_b^T \mathbf{M}_b \mathbf{V}_b + \dot{\boldsymbol{\Phi}}^T \mathbf{M}_m \dot{\boldsymbol{\Phi}} + \mathbf{V}_b^T \mathbf{M}_{bm} \dot{\boldsymbol{\Phi}} + \dot{\boldsymbol{\Phi}}^T \mathbf{M}_{bm} \mathbf{V}_b)$$

$$\mathbf{c}_m = -\frac{1}{2} \frac{\partial}{\partial \boldsymbol{\Phi}} (\mathbf{V}_b^T \mathbf{M}_b \mathbf{V}_b + \dot{\boldsymbol{\Phi}}^T \mathbf{M}_m \dot{\boldsymbol{\Phi}} + \mathbf{V}_b^T \mathbf{M}_{bm} \dot{\boldsymbol{\Phi}} + \dot{\boldsymbol{\Phi}}^T \mathbf{M}_{bm} \mathbf{V}_b)$$

and $\mathbf{q} = [\mathbf{x}_b, \boldsymbol{\Phi}]^T \in \mathbb{S}^{6+n}$ is the state vector of the space robotic manipulator, $\mathbf{M}_{bm} \in \mathbb{R}^{6 \times n}$ is the inertial matrix reflecting the dynamic coupling between the manipulator and the base, $\mathbf{c}_b \in \mathbb{R}^6$ and $\mathbf{c}_m \in \mathbb{R}^n$ are the Coriolis effect terms, and $\mathbf{u} \in \mathbb{R}^n$ is the control torque vector at joints. Equation (8) serves as the environment for the RL training in this work.

3.2. Reinforcement learning

Reinforcement learning algorithms are designed to model decision-making processes using a general MDP [31], where the outcome of an action depends on the current system and the applied action. Our objective is to drive the end-effector of a space robotic manipulator to a desired pose smoothly without colliding with the target and its own links. The relevant state observed from the environment is defined as:

$$\mathbf{s} = [\Phi, \dot{\Phi}, \mathbf{p}_e, \alpha_e, \beta_e, \gamma_e, \dot{\mathbf{p}}_e, \omega_e, \mathbf{p}_T, \alpha_T, \beta_T, \gamma_T, \dot{\mathbf{p}}_T, \omega_T, d, \vartheta] \quad (9)$$

where $\mathbf{s} \in \mathbb{R}^{2n+20}$, subscripts “e” and “T” refer to the end-effector and the target, $(\Phi, \dot{\Phi})$ are the joint angles and angular velocities of the robotic links, $(\mathbf{p}_e, \alpha_e, \beta_e, \gamma_e)$ and $(\mathbf{p}_T, \alpha_T, \beta_T, \gamma_T)$ are the poses of the end-effector and the target, respectively, $(\dot{\mathbf{p}}_e, \omega_e)$ and $(\dot{\mathbf{p}}_T, \omega_T)$ are the velocities of the end-effector and the target, respectively, $d = \|\mathbf{p}_T - \mathbf{p}_e\|$ is the distance difference between the end-effector and the target, and ϑ is the orientation difference between the end-effector and the target.

The actions that control the robot manipulator are the torque input at joints,

$$\mathbf{a} = [u_1, u_2, \dots, u_n]^T \in \mathbb{R}^n \quad (10)$$

Assume the continuous system dynamics between the state $\mathbf{s} \in \mathcal{S}^{2n+20}$ and the action $\mathbf{a} \in \mathcal{A}^n$ is discretized in the time domain at time step t_i , such that,

$$\mathbf{s}_{i+1} = \mathbf{f}(\mathbf{s}_i, \mathbf{u}_i, \mathbf{w}_{s,i+1}) \quad (11)$$

where $\mathbf{u}_i$ is the joint control torque based on the control command action $\mathbf{a}_i$ from a state feedback control policy based on the observed state $\mathbf{o}_i$, and $\mathbf{w}_{s,i} \in \mathcal{N}(0, \mathbf{Q}_{s,i})$ is the noise vector representing environmental perturbation and unmodeled system dynamics. The noise is assumed to obey a zero mean multivariate normal distribution $\mathcal{N}$ with covariance $\mathbf{Q}_{s,i}$. The control input is obtained by a function $\mathbf{g}$ that maps the control command action $\mathbf{a}_i$ output directly from a state-feedback control policy based on the observed state $\mathbf{o}_i$, such that

$$\mathbf{u}_i = \mathbf{g}(\mathbf{a}_i, \mathbf{o}_i, \mathbf{w}_{a,i}) \quad (12)$$

where $\mathbf{w}_{a,i} \in \mathcal{N}(0, \mathbf{Q}_{a,i})$ is the execution error vector, which obeys a zero mean multivariate normal distribution $\mathcal{N}$ with covariance $\mathbf{Q}_{a,i}$.

With the perfect observability assumption, the observed state $\mathbf{o}_i$ is obtained by an observation function $\mathbf{h}$,

$$\mathbf{o}_i = \mathbf{h}(\mathbf{s}_i, t_i, \mathbf{w}_{o,i}) \quad (13)$$

where $\mathbf{o}_i \in \mathcal{O}$ is the observation vector of the system state, $\mathbf{w}_{o,i} \in \mathcal{N}(0, \mathbf{Q}_{o,i})$ is the random processing noise in the state estimation with a zero mean and covariance $\mathbf{Q}_{o,i}$.

The optimal path planning for such a system is to find the best control input $\mathbf{u}_i (i \in \mathbb{Z})$ or an optimal policy π , to maximize the performance function J over a trajectory ρ ,

$$\max_{\rho \sim \pi} J = \max_{\rho \sim \pi} \left[e(\mathbf{s}_0, t_0, \mathbf{s}_N, t_N) + \sum_{i=0}^{N-1} k_i f(\mathbf{s}_i, \mathbf{u}_i, \mathbf{w}_{s,i+1}) \right] \quad (14)$$

where $e(\mathbf{s}_0, t_0, \mathbf{s}_N, t_N)$ is the terminal conditions, k_i is the weight factor, and the trajectory is a sequence of state-action pairs, $\rho = [(\mathbf{s}_0, \mathbf{a}_0), (\mathbf{s}_1, \mathbf{a}_1), \dots, (\mathbf{s}_N, \mathbf{a}_N)]$ satisfying the dynamics. Equation (14) can be recast as an RL agent to maximize a scalar return by learning an optimal policy π^* in interaction with the environment (the robot in the current work) as follows.

Let the policy $\pi: \mathcal{S} \rightarrow \mathcal{A}$ with a set of parameters $\theta \in \mathbb{R}^n$ and express it in the episodic formulation of RL. Then, the optimal policy π^* is the one that maximizes a performance index J in terms of the expectation $\mathbb{E}_{\tau \sim \pi_\theta}$ of the parameter θ^* , which is the discounted sum of rewards obtained along

the trajectory ρ , such that,

$$\begin{aligned} \theta^* &= \underset{\theta}{\operatorname{argmax}} J(\theta) \\ J(\theta) &= \mathbb{E}_{\rho \sim \pi_\theta} \left[\sum_{i=0}^{N-1} \gamma^i R_i(\mathbf{s}_{i+1}, \mathbf{s}_i, \mathbf{a}_i) \right], \gamma \in [0, 1] \end{aligned} \quad (15)$$

where N is the number of time steps in each episode, $R_i(\mathbf{s}_{i+1}, \mathbf{s}_i, \mathbf{a}_i)$ is the immediate reward that maps state-action pairs $(\mathbf{s}_i, \mathbf{a}_i)$ to a scalar value and propagates the state $\mathbf{s}_i$ to the state $\mathbf{s}_{i+1}$ under the action $\mathbf{a}_i$ resulting from the control input $\mathbf{u}_i$. The parameter γ is a discount factor. For long-term planning, the agent treats all future rewards equally by $\gamma = 1$. Conversely, the agent is for short-term planning if $\gamma \ll 1$.

Equation (15) can be solved by the MDP with the stochastic gradient ascent update on the parameter θ [20],

$$\theta_{i+1} = \theta_i + \alpha \nabla_\theta J(\theta_i) \quad (16)$$

where $\alpha \in (0, 1)$ is the learning rate and $\nabla J(\theta_i)$ is the policy gradient.

Note that a differentiable control policy is always available in the path planning problem of a space robotic manipulator. Consequently, the actions involved in the MDP are not stochastic. Then, a deterministic action can be learned directly from given states, instead of learning a set of probability distributions. Accordingly, the DDPG algorithm [32] is employed to estimate the policy gradient $\nabla J(\theta_i)$ as follows.

Let $\mu(\theta)$ be a deterministic policy and $\beta(\theta)$ a behavior policy with trajectories $\beta_\theta(\mathbf{a}|\mathbf{s})$ distinct from the deterministic policy trajectories $\mu_\theta(\mathbf{a}|\mathbf{s})$. Then, the deterministic policy gradient can be estimated by trajectories sampled from a distinct behavior policy $\beta(\mathbf{a}|\mathbf{s})$ based on an Actor-Critic approach [33], such that,

$$\nabla_\theta J(\theta) = \mathbb{E}_{\beta \sim \mu} \left[\sum_{i=0}^{N-1} \nabla_\theta \mu_\theta(\mathbf{a}_i|\mathbf{s}_i) \nabla_a Q_{\mu_\theta,i}(\mathbf{s}_i, \mathbf{a}_i) \right] \quad (17)$$

$$Q_{\mu_\theta,i}(\mathbf{s}, \mathbf{a}) = \mathbb{E}_{\beta \sim \mu} \left[\sum_{i=0}^{N-1} \gamma^i R_i | \mathbf{s} = \mathbf{s}_i, \mathbf{a} = \mu_\theta(\mathbf{s}_i) \right] \quad (18)$$

Here, $Q_{\mu_\theta}(\mathbf{s}, \mathbf{a})$ is the action-value function or Q -function that represents the expected discounted return calculated by the state-action pair $[\mathbf{s}_i, \mathbf{a}_i = \mu_\theta(\mathbf{s}_i)]$ and corresponding rewards based on the policy μ_θ from the i th time step until the end of the episode. The sole dependence of the Q -function on the environment allows for off-policy Q -learning [34] using trajectories generated from a different stochastic behavior policy $\beta(\mathbf{a}|\mathbf{s})$. Based on the greedy policy $\mu(\mathbf{s}) = \underset{\mathbf{a}}{\operatorname{argmax}} Q_{\mu_\theta}(\mathbf{s}, \mathbf{a})$, the Q -function is approximated by the Bellman equation [35],

$$Q_{\mu_\theta}(\mathbf{s}_i, \mathbf{a}_i) = \mathbb{E}_{\beta \sim \mu} [R(\mathbf{s}_i, \mathbf{a}_i) + \gamma Q_{\mu_\theta}(\mathbf{s}_{i+1}, \mu(\mathbf{s}_{i+1}))] \quad (19)$$

Then, the Q -function can be evaluated recursively over a parameter θ^Q using temporal difference learning,

$$\begin{aligned} Q_{\mu_\theta}^{i+1}(\mathbf{s}_i, \mathbf{a}_i) &= Q_{\mu_\theta}^i(\mathbf{s}_i, \mathbf{a}_i) + \alpha \left\{ \mathbb{E}_{\beta \sim \mu} [R_i(\mathbf{s}_i, \mathbf{a}_i) + \gamma Q_{\mu_\theta}(\mathbf{s}_{i+1}, \mu(\mathbf{s}_{i+1}))] \right. \\ &\quad \left. - Q_{\mu_\theta}^i(\mathbf{s}_i, \mathbf{a}_i) \right\} \end{aligned} \quad (20)$$

This learning is achieved by minimizing the loss function $L(\theta^Q)$:

$$\begin{aligned} L(\theta^Q) &= \mathbb{E}_{\beta \sim \eta} [(Q_{\mu_\theta,i}(\mathbf{s}_i, \mathbf{a}_i) - y_i)^2] \\ y_i &= R_i + \gamma^i Q_{\mu_\theta,i}(\mathbf{s}_{i+1}, \mu(\mathbf{s}_{i+1})) | \theta^Q \end{aligned} \quad (21)$$

3.3. DDPG algorithm implementation

Applying Q -learning to continuous systems like the robotic manipulator is challenging because it requires optimizing actions $\mathbf{a}_i$ at each time step. Instead, a Deep Q-Network (DQN) approach [32] is used to

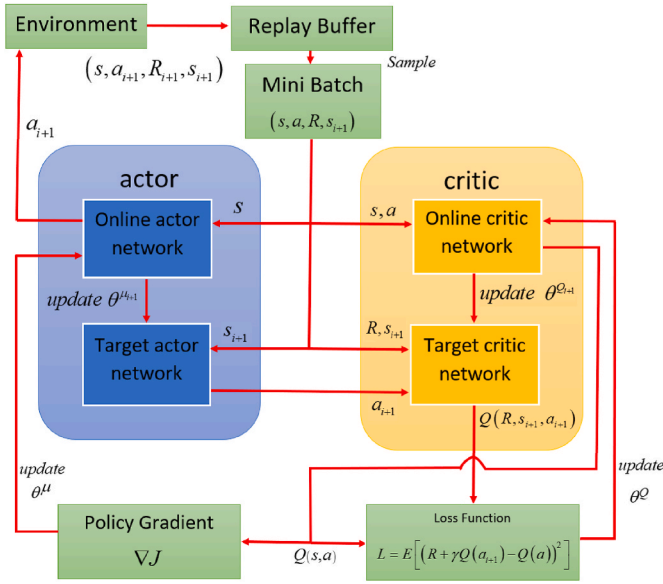


Fig. 2. Implementation chart of DDPG algorithm.

approximate the Q -function by Deep Neural Networks (DNN) with state and action as input and expected cumulative reward as output. The DQN algorithm involves two DNNs: main and target network. The DDPG

extends this two DNN into Actor and Critic. The Actor is split into online and target actor networks, and the Critic is split into online and target networks as shown in Fig. 2. The Actor returns actions deterministically in a continuous space based on its interaction with the robotic manipulator (environment) via the parameterized policy $\mu_{\theta^{\mu}}$, and is updated by gradient ascent on the policy gradient in Eq. (16). The Critic network evaluates the agent's performance by estimating the Q -value of state-action pairs (s_i, a_i) using Eq. (19) to guide the generation of next action. To ensure the samples are independently distributed, a replay buffer is used to save the tuple (s_{i+1}, s_i, a_i, R_i) in stacks. The tuples are sampled from the environment in accordance with the exploration policy. At each time step, Actor and Critic are updated by uniformly sampling a minibatch from the buffer. Finally, to improve the Q -learning

Table 3
Reinforcement learning parameters.

Parameter	VALUE
Learning rate	0.001
Replay buffer size	50,000
Discount factor for future reward	0.99
Smooth factor	0.001
Minibatch size	128
Number of episodes	5000
Episode timesteps	400
Timestep size (seconds)	0.025
Episode time (seconds)	10

Table 2

DDPG algorithm pseudo codes [36].

Procedure:

Randomly initialize online actor $\mu(s|\theta^{\mu})$ and critic $Q(s, a|\theta^Q)$ networks by respective weights (θ^{μ}, θ^Q) .

Reset target actor μ' and critic Q' networks with weights $\theta^{\mu} \rightarrow \theta^{\mu'}$, $\theta^Q \rightarrow \theta^{Q'}$.

Reset replay buffer (RAM).

For each episode M **do**:

For each timestep:

Select an action $a = \mu(s|\theta^{\mu}) + \mathbb{N}$, where s and $\mathbb{N}$ are current state and random process exploration noise.

Execute action a , observe reward R & new state s_{i+1} .

Store the experience (s, a, R_i, s_{i+1}) in the replay buffer.

Sample random minibatch of experiences (s, a, R_i, s_{i+1}) from the replay buffer.

If s_{i+1} is the terminal state, set the value function target as

$y = R$

Else

set the value function target as

$y = R + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$

Update critic parameters by minimizing the loss L on all sampled experiences,

$$L = \frac{1}{M} \sum_{i=1}^M [y - Q(s, a|\theta^Q)]^2$$

Update actor parameters to maximize the expected discounted reward by the sampled policy gradient,

$$\nabla_{\theta^{\mu}} J \approx \frac{1}{M} \sum_{i=1}^M \nabla_a Q(s, a|\theta^Q) \nabla_{\theta^{\mu}} \mu(s|\theta^{\mu})$$

Update target actor and critic parameters by:

$$\theta^{Q'} = \tau \theta^Q + (1 - \tau) \theta^{Q'} \text{ and } \theta^{\mu'} = \tau \theta^{\mu} + (1 - \tau) \theta^{\mu'}$$

End for

End for

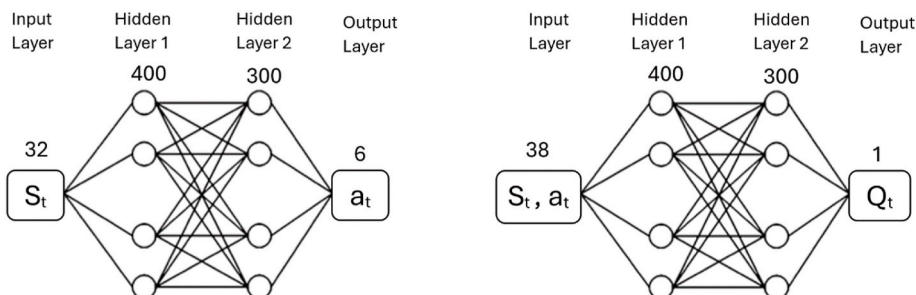


Fig. 3. Illustration of Actor and Critic networks.

Table 4
Parameters of 6-DOF robotic manipulator.

Items	Mass (kg)	Length (m)	I _x (kg·m ²)	I _y (kg·m ²)	I _z (kg·m ²)
Base	1700	0.75 Cube	1434	1434	1735
Link 1	5	0.25	0.0292	0.0292	0.0063
Link 2	5	0.2	0.0292	0.0292	0.0063
Link 3	50	0.75	0.0625	26.1	26.1
Link 4	10	0.2	0.0125	0.215	0.215
Link 5	50	0.75	0.0625	26.1	26.1
Link 6	5	0.2	0.0063	0.0292	0.0292

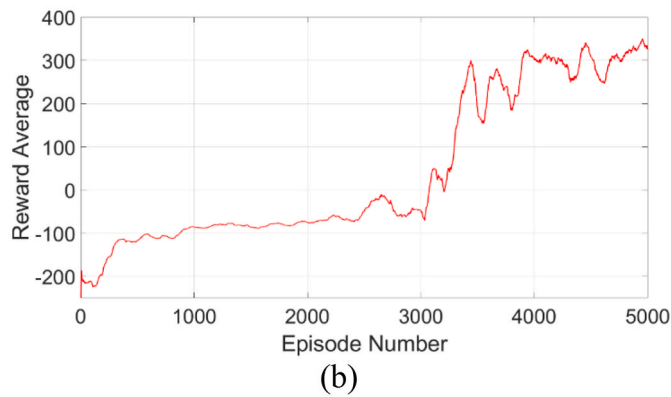
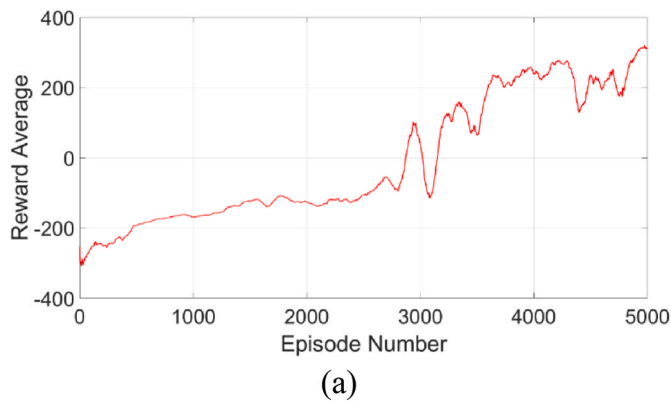


Fig. 4. RL training process: (a) without velocity constraint, and (b) with velocity constraint.

stability, the following update schemes are introduced for the target Actor and Critic parameters by

$$\theta_{Q,i+1} = \Gamma \theta_{Q,i} + (1 - \Gamma) \theta_{Q,i+1} \text{ and } \theta_{\mu,i+1} = \Gamma \theta_{\mu} + (1 - \Gamma) \theta_{\mu,i+1} \quad (22)$$

where $\Gamma \ll 1$ is the smoothing factor.

The implementation and corresponding pseudo codes of the DDPG algorithm are shown in Table 2 and Fig. 2.

For the 6-DOF robotic manipulator ($n = 6$), the dimensions of state and action spaces are $\mathbf{s} \in \mathbb{S}^{32}$ and $\mathbf{a} \in \mathbb{A}^6$, respectively. Thus, the Actor and Critic DNNs are constructed by standard feed-forward neural networks with two hidden layers as shown in Fig. 3, where the hidden layers 1 and 2 comprise 400 and 300 neurons. The input and output of the Actor network are state and action, while the input and output of the Critic network are state-action pair and action value.

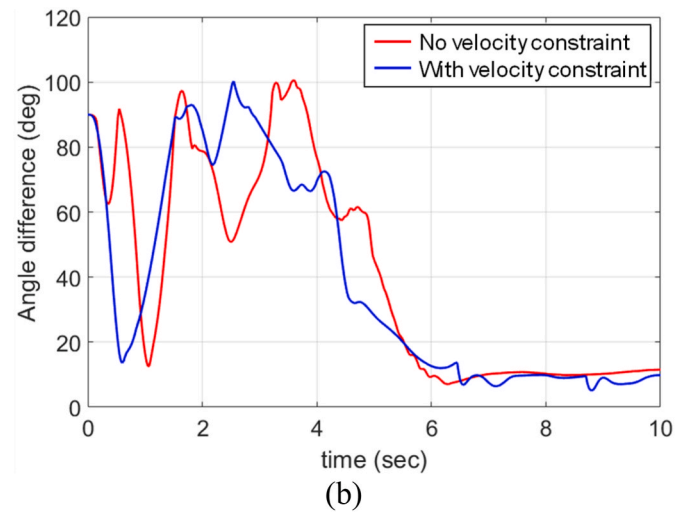
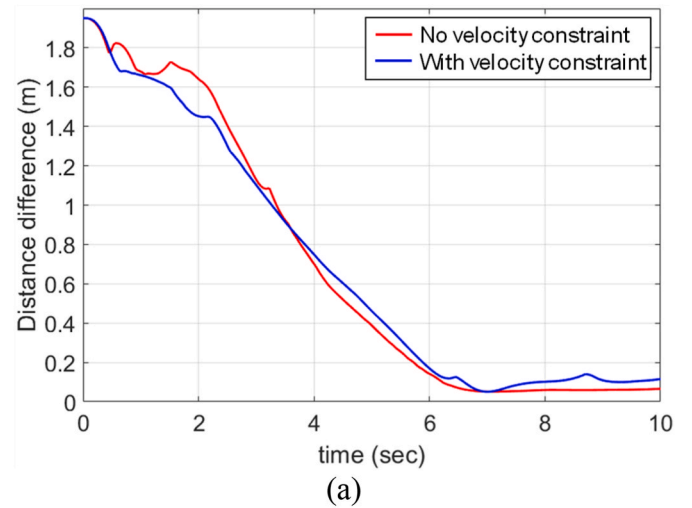


Fig. 6. Tracking errors of position (a) and orientation (b).

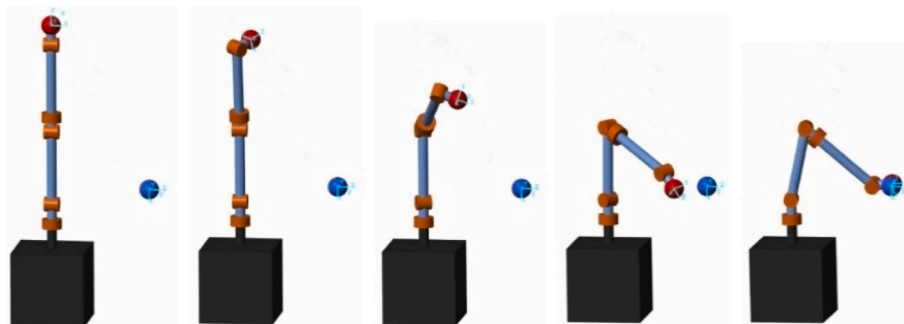


Fig. 5. Snapshots of robotic manipulator reaching the target without velocity constraint.

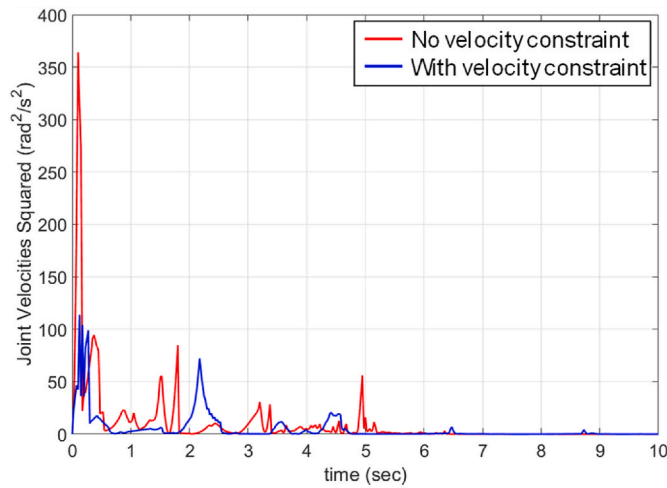


Fig. 7. Squared sums of all joint velocities with and without velocity constraint.

3.4. Reward function

The objective of path planning is to find an optimized path for the end-effector to precisely align with the target's pose smoothly with minimal fluctuations in joint velocities, while preventing the end-effector from colliding with the target or other links of the manipulator and avoiding dynamic singularities in the configuration space of the manipulator. Accordingly, the reward function must include following parts to reflect these requirements.

The primary part of the reward function is focusing on aligning the end-effector's pose with the target's pose by minimizing the pose error quickly as follows:

$$R = -K_d d - K_\theta \vartheta + r_d + r_\theta \quad (23)$$

where $d = \|\mathbf{p}_T - \mathbf{p}_e\|$ is the position error and $\vartheta = 2 \arccos[\text{Re}(\xi_T - \xi_e)]$ denotes the orientation error expressed in terms of quaternions $\xi_T - \xi_e = \xi_T \times \xi_e^*$ [37]. Here, ξ_e and ξ_T are the unit quaternions representing the

orientation of the end-effector and the target, respectively, while ξ_e^* is the conjugate of ξ_e . Moreover, K_d and K_θ are positive reward coefficients for the position and orientation errors, respectively, to minimize position and orientation errors, respectively. Furthermore, as the pose error (d, ϑ) decreases to below a pre-defined tolerance, the reward R also approaches to zero. To achieve the desired learning outcome in such condition, the reward function is designed to have large positive rewards for actions that compel the agent stay in the close proximity of the target, thereby ensuring the end-effector maintains its desired pose for grasping the target throughout the remaining time steps. Consequently, the large positive rewards (r_d, r_θ) in the reward function are activated when the pose error (d, ϑ) falls below the pre-defined tolerances, such that,

$$r_d = \begin{cases} 0 & \text{if } d \geq d_0 \\ r_d & \text{if } d < d_0 \end{cases}, r_\theta = \begin{cases} 0 & \text{if } \vartheta \geq \vartheta_0 \cup d \geq d_0 \\ r_\theta & \text{if } \vartheta < \vartheta_0 \cap d < d_0 \end{cases} \quad (24)$$

where d_0 and ϑ_0 are the predefined tolerances for the position and orientation errors of the end-effector, respectively. This incentive reward is crucial for guiding the agent towards achieving precise alignment and maintaining proximity to the target, facilitating successful task execution.

The secondary part of the reward function is tailored to ensure a smooth trajectory of the end-effector with minimal fluctuations in joint

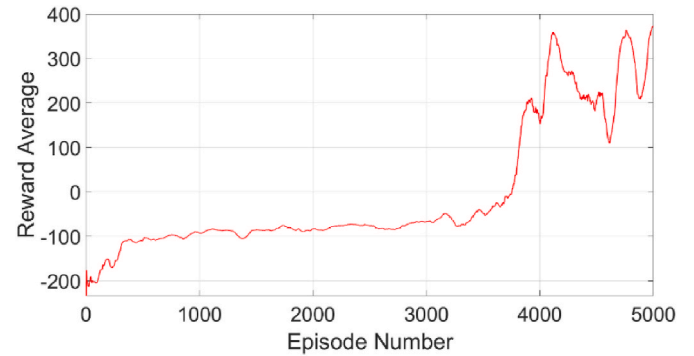


Fig. 9. RL training process.

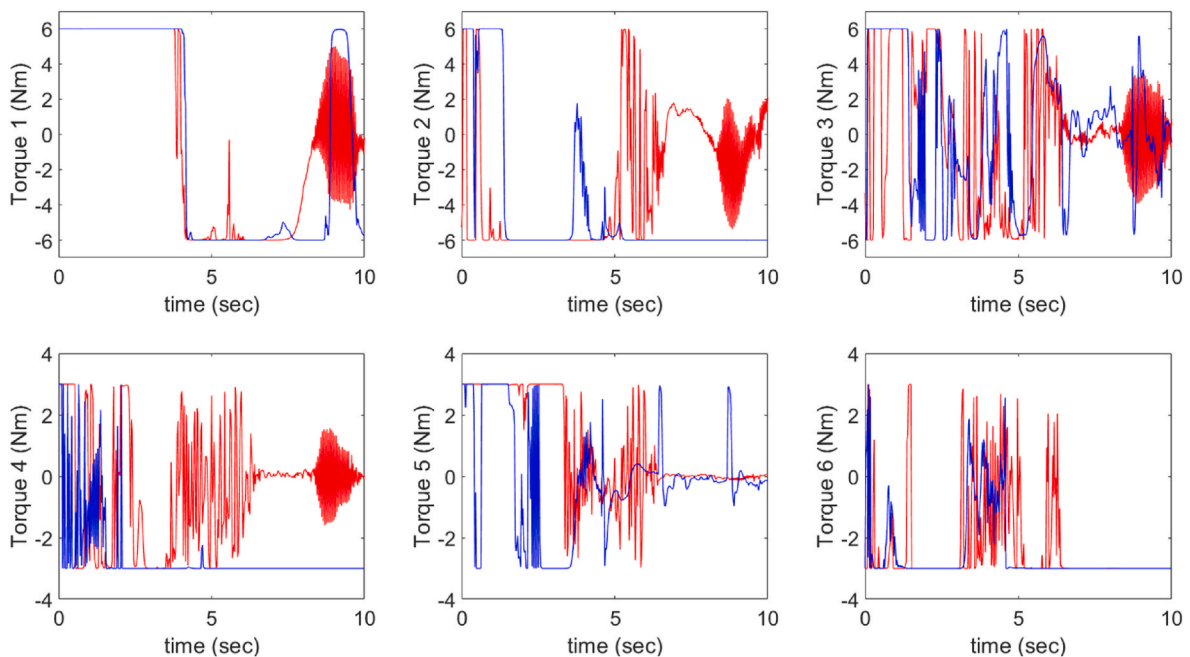


Fig. 8. Time histories of joint torques with and without velocity constraint.

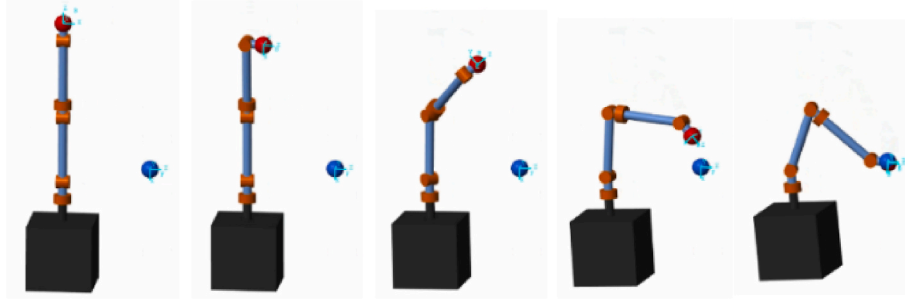


Fig. 10. Snapshots of robotic manipulator reaching the target at higher mass ratio.

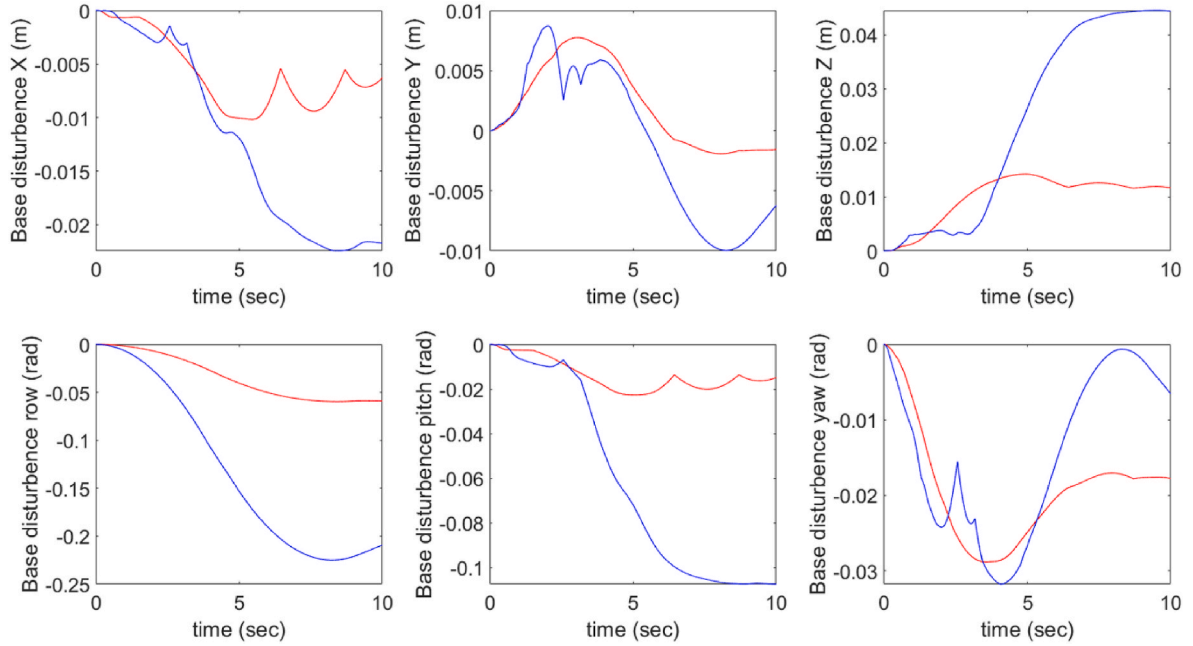


Fig. 11. Position and attitude disturbances of base spacecraft: — small mass ratio, — high mass ratio.

velocities, while avoiding any collisions with both the target and other links of the manipulator. This objective is achieved by adding a negative reward that is proportional to the magnitude of the square sum of the joint velocities like $-p_j \sum (\dot{\varphi})^2$ to punish abrupt joint movements. Moreover, a large negative reward is applied to penalize the agent whenever the distance between the end-effector and either the target or other links of the manipulator is less than a predefined safety threshold. Thus, the final reward function is defined as follows:

$$R = -K_d d - K_\theta \vartheta + r_d + r_\theta - p_c - p_j \sum (\dot{\varphi})^2 \quad (25)$$

where $p_j > 0$ is the penalty coefficient for smoothing the joint velocities and $p_c > 0$ is the penalty for collision avoidance. The latter is activated when the distance between the end-effector and the target or any manipulator link is less than the collision safety distance d_c , as follows:

$$p_c = \begin{cases} 0 & \text{if } \min_{i \neq j} \|\mathbf{r}_i - \mathbf{r}_j\| \geq d_c \\ p_c & \text{if } \min_{i \neq j} \|\mathbf{r}_i - \mathbf{r}_j\| < d_c \end{cases} \quad (26)$$

3.5. Observation noise

Perfect observation of the state is unattainable in real-world scenarios. Accordingly, the agent should be trained by noisy observations $(\mathbf{p}_T, \xi_T, d, \vartheta)$, such that,

$$(\mathbf{p}_T, \xi_T, d, \vartheta)_{out} = (1 + R_n)(\mathbf{p}_T, \xi_T, d, \vartheta)_{in} \quad R_n \in \mathbb{N}(0, 1) \quad (27)$$

where $(\mathbf{p}_T, \xi_T, d, \vartheta)_{in}$ is the ideal target pose and associated error, $(\mathbf{p}_T, \xi_T, d, \vartheta)_{out}$ is the noisy pose and associated error, and R_n is the signal-to-noise ratio (SNR), which is assumed 10/1 in the current study.

4. Results and discussion

The proposed RL approach is applied to the path planning for a 6-DOF free-floating robotic manipulator to demonstrate its efficacy and robustness. The training and experiments were conducted by simulation using MATLAB® and Simulink®. Detailed information regarding the RL training parameters is given in Table 3.

The detailed physical parameters of the manipulator are outlined in Table 4 as per [38]. The initial pose of the end-effector is $(x, y, z, \xi)_e = (0, 0, 2.675, -1, 0, 0, 0)$ and the target's pose is $(x, y, z, \xi)_T = (1, 0, 1, -0.707, 0, -0.707, 0)$. The torque limits for the first three joints are set at ± 6 Nm, while the remaining three joints have a constraint of ± 3 Nm. The study considers two scenarios regarding the manipulator-base spacecraft mass ratio. The first scenario is roughly at 7.4 %, which is typical in known space robotic missions [1]. The second scenario is at 30 % to assess the performance of the proposed reward function in the presence of strong dynamic coupling between the robotic manipulator and the base spacecraft, which is prone to dynamic singularities. The

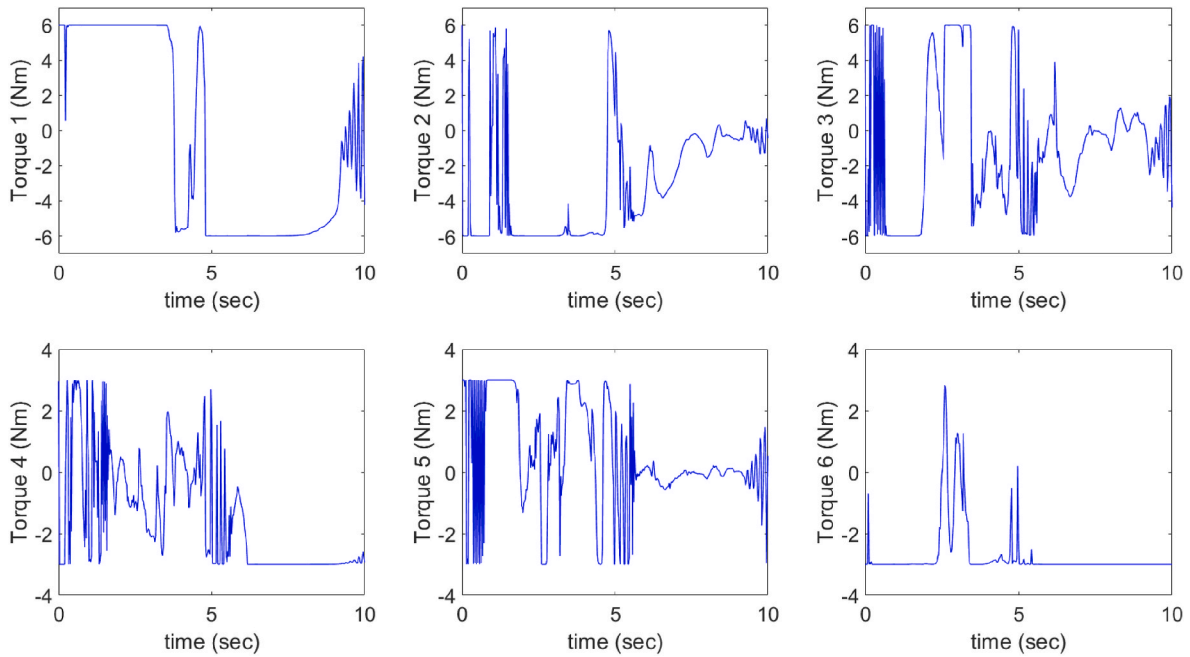


Fig. 12. Time histories of joint torques generated by the agent.

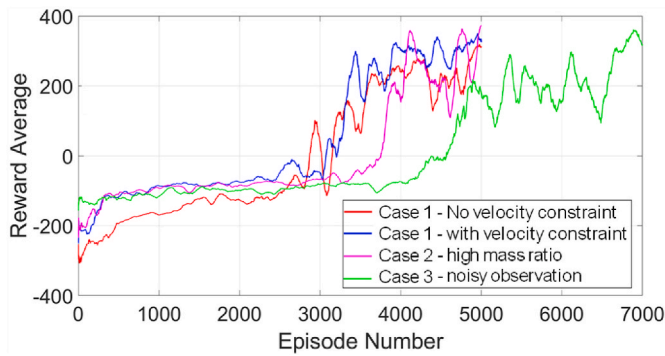


Fig. 13. RL training process.

parameters used in the reward function are defined as $k_d = 0.1$, $k_\theta = 0.001$, $d_0 = 0.15m$, $\theta_0 = 15^\circ$, $r_d = 1$, $r_\theta = 2$, $p_j = 0.005$, $p_c = 0.5$, and $d_c = 0.3m$.

4.1. Case 1 – approach target without and with velocity control

In this case, the agent is trained to guide the end-effector towards the desired pose $(x, y, z, \xi)_T$ with a typical manipulator-base spacecraft mass ratio of 7.4 % with and without imposing any constraints on joint velocities. The training process, derived from 5000 episodes, is shown in Fig. 4. Initially, the agent starts its learning process by searching for the highest reward on actions, primarily focusing on the reward term $(-K_d d - K_\theta \theta)$. Around the halfway, after approximately 2500 episodes, the end-effector starts to consistently reach the target's pose. Once the end-effector approaching to the target's pose with errors less than $d_0 = 0.15m$ and $\theta_0 = 15^\circ$, the agent relies on the activation of large rewards (r_d, r_θ) to get the end-effector closer to the target's pose and subsequently maintain in that pose throughout the remaining timesteps by concentrating on maximizing rewards within this specific region, thereby limiting its search space. The training process eventually converges after 3700 episodes without the velocity constraint, indicating the agent's ability in learning and executing the task effectively and efficiently. When the velocity constraint is considered, the training process takes

slightly longer time, with a convergence being achieved after approximately 4000 episodes.

Fig. 5 depicts a series of snapshots of the robotic manipulator in its approach towards the target without any velocity constraints. It shows the movement of the base spacecraft in the entire process is very small due to the low manipulator-base spacecraft mass ratio. The behavior of the robotic manipulator trained with velocity constraint is similar and is not depicted here for brevity. The position (d) and orientation (θ) tracking errors between the end-effector and the target over time are shown in Fig. 6. Evidently, the trained agent steers the end-effector toward the target's pose within the defined tolerance range in 6s, and subsequently, the end-effector stays within that region for the remaining duration. It is worth mentioning that the threshold $d_0 = 0.15m$ does not imply that the minimum distance error the trained agent can reach; instead, it signifies that crossing this threshold secures a large reward r_d . In fact, the agent achieves a minimum distance error of 0.05m. The threshold helps the agent to swiftly approach this region at an earlier stage, then spend the rest of the time steps searching for more rewards due to its greedy nature, effectively refining the search space for optimal solutions. Similarly, the same approach applies to the orientation error, achieving a minimum orientation difference of 5.0° , shown in section 4.4.

The impact of implementing a velocity constraint in the training process is shown Fig. 7 by comparing the squared sums of all joint velocities ($\sum (\dot{\phi})^2$) for scenarios with and without velocity constraint. The trajectory executed by the agent trained without any velocity constraints is represented by the red line, while the trajectory from the agent trained with the velocity constraint incorporated into the reward function is depicted by the blue line. It is clearly shown that the velocity constraint added in the reward function had reduced the squared sum of all joint velocities by more than three times. This evident decrease confirms that incorporating the velocity constraint into the reward function successfully smooths the robotic manipulator's motion. The ability to refine the motion of the agent through a customized reward function paves the way for further investigations into reward function design. Future studies could explore the inclusion of various criteria within the reward function that have been previously utilized in optimization efforts, such as the minimization of torques, exerted forces, energy consumption, and disturbances to the base, among others.

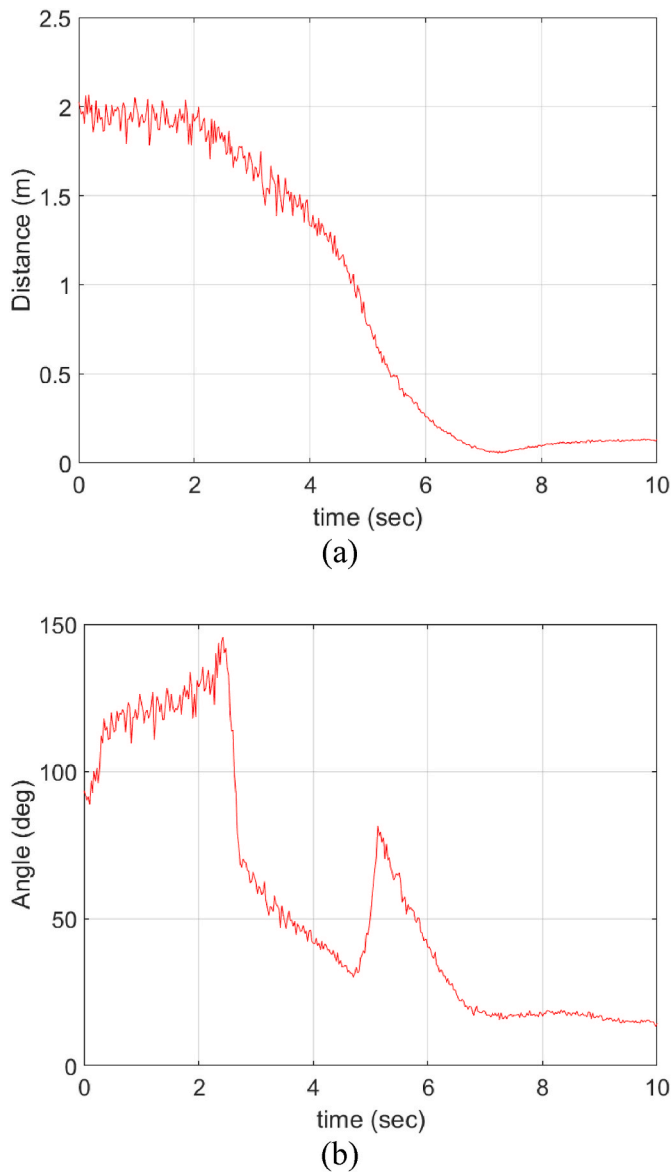


Fig. 14. Tracking errors: (a) position d and (b) orientation θ .

Finally, Fig. 8 shows the time histories of actions or torques at each joint given by the agent to achieve the pose alignment. The torques generated by the agent trained without any velocity constraints are represented by the red line, while the torques generated by the agent trained with the velocity constraint incorporated into the reward function are depicted by the blue line. This comparison shows clearly that the torques from the agent trained with the velocity constraint exhibit smoother characteristics and fewer oscillations in comparison with the torques generated by the agent without any velocity constraints. Additionally, the agent with the velocity constraint achieves stable torques at the end of training, indicating a more controlled and consistent performance throughout the operation.

4.2. Case 2 – approach target with a heavier manipulator

In this case, the agent is trained to guide the end-effector towards the target's pose $(x, y, z, \xi)_T$ while dealing with an increased manipulator-base spacecraft mass ratio of 30 %. As a result, adjustments are made to the base mass and inertia values listed in Table 4, with mass becoming 425 kg and the inertia components (I_x, I_y, I_z) revised to (358, 358, 434) $\text{kg}\cdot\text{m}^2$, respectively. Aside from the altered mass ratio, the geometric

parameters of the manipulator and other RL parameters remain unchanged, including the incorporation of the velocity constraint in the reward function. The training process is shown in Fig. 9. It reveals that the higher mass ratio prolongs the time needed for the end-effector to converge towards the target's pose. Specifically, approximate 3700 episodes are required before the end-effector starts to reach the target pose, which shows an increase from the 3200 episodes observed in the previous case illustrated in Fig. 4(b), due to the stronger dynamic coupling between the manipulator and the base spacecraft.

Fig. 10 presents a sequence of snapshots capturing the robotic manipulator as its end-effector approaches the target's pose. These images clearly show that the base spacecraft experiences significant disturbances in the process as a result of the significant dynamic coupling between the manipulator and the base spacecraft due to the high mass ratio between them. To further illustrate the effects of dynamic coupling, the translational and rotational movements or disturbances of the base spacecraft observed during the approaching process are shown in Fig. 11. For a comparative analysis, the movements of the base spacecraft from the previous case with a smaller mass ratio of 7.4 % are also plotted in Fig. 11, where red lines represent the small mass ratio case, and blue lines denote the high mass ratio case. This comparison reveals significant movements in the base spacecraft in case of high mass ratio. Additionally, Fig. 12 shows the time histories of torques at each joint generated by the agent to achieve the target's pose. Compared with the observations in Fig. 8, it is evident that the higher mass ratio intensifies the dynamic coupling between the manipulator and the base spacecraft, leading to more torque oscillations. At the end of training, the torques at four out of the six joints have not yet stabilized and torque oscillation is evident in Fig. 14 and more pronounced in Fig. 15, where noise is introduced into the state observations. This phenomenon, referred to as action oscillation in Ref. [39], arises from the balance between exploration and exploitation during training. Deep reinforcement learning algorithms are particularly prone to this issue in discrete action settings, where an agent may select different actions in consecutive steps despite only slight differences in states. The problem is exacerbated by the presence of noise in the state observations. Although this oscillation is undesirable, the agent perceives it as the optimal strategy to maximize rewards, given that its decisions are driven by a reward only. Future work is needed to mitigate this phenomenon by incorporating a penalty for excessive changes in actions or torque. These findings highlight the effectiveness of RL in the path planning of free-floating robotic manipulators, even in scenarios with a high mass ratio. They suggest that future robotic missions might not require base spacecraft of greater mass to minimize the effects of dynamic coupling, paving the way for the use of lighter base spacecraft. This advancement could enhance efficiency, flexibility, and cost-effectiveness in space missions.

4.3. Case 3 – approach target with noisy observation

In this case, the influence of state measurement noise on RL is studied by introducing white noises into state observation at a signal-to-error ratio of 10/1 as shown in Eq. (27). The mass ratio between the robotic manipulator and the base spacecraft is set to 7.4 %. The parameters of the manipulator and RL remains the same as those in Case 1 with the incorporation of joint velocity constraint in the reward functions. Fig. 13 shows the result of training together with the training processes in the previous two cases. It is noteworthy that the training process requires more episodes to converge for noisy observations, specifically 7000 as compared to 5000 in previous cases. Nevertheless, the agent effectively overcomes the challenges posed by noisy inputs, which demonstrates the effectiveness and robustness of the proposed reward function.

Next, the position (d) and orientation (θ) tracking errors between the end-effector and the target over time are shown in Fig. 14. While these tracking errors exceed those observed in previous cases shown in Fig. 6, the agent nonetheless successfully aligns the end-effector with the

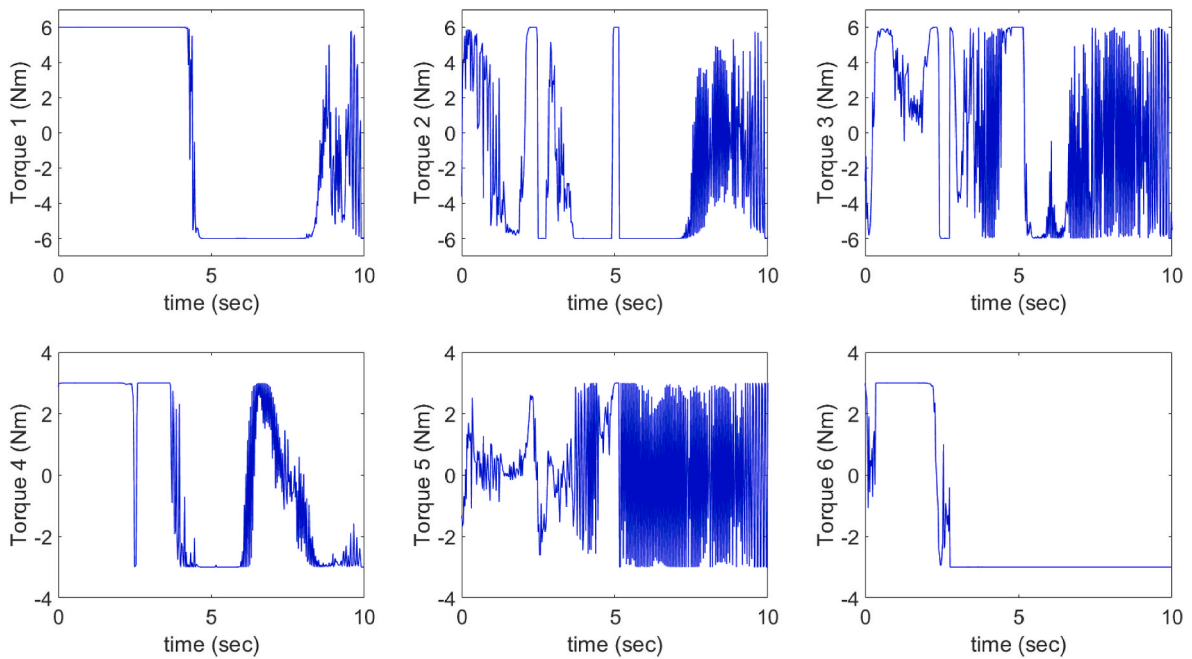


Fig. 15. Time histories of joint torques by the agent trained with noisy observation.

Table 5
Comparison among all simulation cases.

Simulation case	Comparison metrics		
	Episodes starting to converge	Position tracking error (meter)	Orientation tracking error (deg)
Case 1 – no velocity constraint	2869	0.0513	7.0795
Case 1 – with velocity constraint	3072	0.0509	5.0899
Case 2 – high mass ratio + velocity constraint	3751	0.0192	5.8835
Case 3 – noisy observations + velocity constraint	4567	0.0567	13.3055

target’s pose. Furthermore, Fig. 15 shows the time histories of joint torques generated by the agent, which are required to reach the target’s pose. Compared to the torque patterns in Fig. 8, a significant increase in torque oscillations is observed. This phenomenon can be attributed to the impact of noisy observations. This torque oscillations do not disappear even when the pose of the end-effector is aligned with the target’s pose because the agent continuously adjusts the torques in order to maintain a stable pose of the end-effector amidst the perturbations induced by noise in the observed states, as shown in Fig. 14.

4.4. Summary

The outcomes of all simulations conducted are summarized in Table 5, which includes a comparison based on the required training episodes necessary for convergence and the minimum recorded tracking errors for both position and orientation of the end-effector that were successfully achieved.

5. Conclusion

This study investigated the path planning for 6-DOF free-floating space robotic manipulators through DDPG-based Reinforcement Learning. A new reward function was designed to address several key

aspects in path planning: ensuring the end-effector’s pose aligns with the target’s, avoiding collisions with the target and the manipulator’s other links, smoothing joint velocities of the manipulator, managing the significant variations in mass ratio between the manipulator and the base spacecraft, and increasing the robustness of the path planning with noise in the state observations. Furthermore, this new reward function utilizes quaternions instead of Euler angles for orientation control, which helps minimize pose misalignments between the end-effector and the target and avoid singularities. The results demonstrated that the RL algorithm, with a well-designed reward function incorporating these considerations, can achieve the desired outcomes more efficiently. Moreover, the RL algorithm can easily handle the significant dynamic coupling between the manipulator and the base spacecraft in cases with a high mass ratio. Even when trained with noisy state observations, the agent successfully completes the path planning task, highlighting the RL algorithm’s ability for real-world mission design where the noise in observation is inevitable. This study highlights the significance of reward function design on training effectiveness and solution quality, laying a solid foundation for the development of free-floating space robotic missions. Future study will explore additional challenges, such as navigating obstacles between the end-effector and the target and tracking tumbling targets, which is vital for practical missions.

CRediT authorship contribution statement

Ahmad Al Ali: Conceptualization, Formal analysis, Investigation, Methodology, Validation, Visualization, Writing – original draft, Writing – review & editing. Jian-Feng Shi: Conceptualization, Investigation, Methodology, Writing – review & editing, Visualization. Zheng H. Zhu: Conceptualization, Formal analysis, Funding acquisition, Investigation, Methodology, Project administration, Resources, Supervision, Validation, Visualization, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work is supported by the FAST grant (19FAYORA14) of the Canadian Space Agency and the Discovery Grant (RGPIN-2018-05991) and CREATE SMART-ART program (555425-2021) of the Natural Sciences and Engineering Research Council of Canada.

References

- [1] F.-A. Angel, O. Ma, K. Pham, S. Ulrich, A review of space robotics technologies for on-orbit servicing, *Prog. Aerosp. Sci.* 68 (2014) 1–26, <https://doi.org/10.1016/j.paerosci.2014.03.002>.
- [2] B.P. Larouche, Z.H. Zhu, Autonomous robotic capture of the non-cooperative target using visual servoing and motion predictive control, *Auton. Robots* 37 (2) (Jan. 2014) 157–167, <https://doi.org/10.1007/s10514-014-9383-2>.
- [3] <https://www.asc-csa.gc.ca/eng/blog/2021/04/16/canadarm2-celebrates-20-years-on-international-space-station.asp>, visited on December 9, 2023.
- [4] S. Dubowsky, E. Papadopoulos, The kinematics, dynamics, and control of free-flying and free-floating space robotic systems, *IEEE Trans. Robot. Autom.* 9 (5) (1993) 531–543, <https://doi.org/10.1109/70.258046>.
- [5] F. Pfeiffer, Manipulator trajectory planning and control, *IFAC Proc* 19 (14) (1986) 325–330, [https://doi.org/10.1016/S1474-6670\(17\)59499-9](https://doi.org/10.1016/S1474-6670(17)59499-9).
- [6] L. Wang, X. Lai, Q. Meng, M. Wu, Effective control method based on trajectory optimization for three-link vertical underactuated manipulators with only one active joint, *IEEE Trans. Cybern.* 53 (6) (2023) 3782–3793, <https://doi.org/10.1109/TCYB.2021.3125187>.
- [7] P. Tsiotras, M. King-Smith, L. Ticozzi, Spacecraft-mounted robotics, *Annu. Rev. Control Robot. Auton. Syst.* 6 (No.1) (2023) 335–362, <https://doi.org/10.1146/annurev-control-062122-082114>.
- [8] J. Ratajczak, K. Tchoń, Coordinate-free jacobian motion planning: a 3-D space robot, *IEEE Trans. Syst. Man Cybern. Syst.* 52 (8) (2022) 5354–5361, <https://doi.org/10.1109/TSMC.2021.3125276>.
- [9] K. Nanos, E.G. Papadopoulos, On the dynamics and control of free-floating space manipulator systems in the presence of angular momentum, *Front. Robot. AI* 4 (26) (2017), <https://doi.org/10.3389/frobt.2017.00026>.
- [10] T. Rybus, K. Seweryn, J.Z. Sasiadek, Control system for free-floating space manipulator based on nonlinear model predictive control (NMPC), *J. Intell. Robot. Syst.* 85 (3–4) (2017) 491–509, <https://doi.org/10.1007/s10846-016-0396-2>.
- [11] Y. Dai, C. Xiang, Y. Zhang, Y. Jiang, W. Qu, Q. Zhang, A review of spatial robotic arm trajectory planning, *J. Aerosp.* 9 (7) (2022) 361, <https://doi.org/10.3390/aerospace9070361>.
- [12] S.K. Agrawal, R. Garimella, G. Desmier, Free-floating closed-chain planar robots: kinematics and path planning, *Nonlinear Dyn.* 9 (1996) 1–19, <https://doi.org/10.1007/BF01833290>.
- [13] J. Luo, M. Yu, M. Wang, J. Yuan, A fast trajectory planning framework with task-priority for space robot, *Acta Astronaut.* 152 (2018) 823–835, <https://doi.org/10.1016/j.actaastro.2018.09.023>.
- [14] W. Xu, L. Bin, Y. Xu, Practical approaches to handle the singularities of a wrist-partitioned space manipulator, *Acta Astronaut.* 68 (1–2) (2011) 269–300, <https://doi.org/10.1016/j.actaastro.2010.07.004>.
- [15] M. Wang, J. Luo, U. Walter, Trajectory planning of free-floating space robot using particle swarm optimization (PSO), *Acta Astronaut.* 112 (2015) 77–88, <https://doi.org/10.1016/j.actaastro.2015.03.008>.
- [16] X. Shao, G. Sun, C. Xue, X. Li, Nonsingular terminal sliding mode control for free-floating space manipulator with disturbance, *Acta Astronaut.* 181 (2021) 396–404, <https://doi.org/10.1016/j.actaastro.2021.01.038>.
- [17] X. Ye, Z.-H. Dong, J.-C. Hong, Research on adaptive reaction null space planning and control strategy based on VFF-RLS and SSADE-ELM algorithm for free-floating space robot, *Electronics* 8 (article) (2019) 1111, <https://doi.org/10.3390/electronics8101111>.
- [18] Z. Xie, T. Sun, T.H. Kwan, Z. Mu, X. Wu, A new reinforcement learning based adaptive sliding mode control scheme for free-floating space robotic manipulator, *IEEE Access* 8 (2020) 127048–127064, <https://doi.org/10.1109/ACCESS.2020.3008399>.
- [19] H. Nguyen, H. La, Review of Deep reinforcement learning for robot manipulation, *Third IEEE Int. Conf. Robot. Comp. (IRC)* (2019) 590–595, <https://doi.org/10.1109/IRC.2019.00120>. Naples, Italy.
- [20] R.S. Sutton, A.G. Barto, *Reinforcement Learning: an Introduction*, second ed., MIT Press, Cambridge, MA, USA, 2018, pp. 2–25.
- [21] Yan, Q. Zhang, Z. Liu, X. Wang, B. Liang, Control of free-floating space robots to capture targets using soft Q-learning, in: 2018 IEEE Int. Conf. Robot. Biomim. (ROBIO), 2018, pp. 654–660, <https://doi.org/10.1109/ROBIO.2018.8665049>. Kuala Lumpur, Malaysia.
- [22] Y.-C. Liu, C.-Y. Huang, DDPG-based adaptive robust tracking control for aerial manipulators with decoupling approach, *IEEE Trans. Cybern.* 52 (8) (2022) 8258–8271, <https://doi.org/10.1109/TCYB.2021.3049555>.
- [23] X. Hu, X. Huang, T. Hu, Z. Shi, J. Hui, MRDDPG algorithms for path planning of free-floating space robot, in: 2018 IEEE 9th Int. Conf. Softw. Eng. Service Science (ICSESS), 2018, pp. 1079–1082, <https://doi.org/10.1109/ICSESS.2018.8663748>. Beijing, China.
- [24] D. Du, Q. Zhou, N. Qi, X. Wang, Y. Liu, Learning to control a free-floating space robot using Deep reinforcement learning, 2019 IEEE Int. Conf. Unmann. Syst. (ICUS) (2019) 519–523, <https://doi.org/10.1109/ICUS48101.2019.8995991>. Beijing, China.
- [25] Y. Li, X. Hao, Y. She, S. Li, M. Yu, Constrained path planning of free-float dual-arm space manipulator via Deep reinforcement learning, *Aerosp. Sci. Technol.* 109 (2021) 106446, <https://doi.org/10.1016/j.ast.2020.106446>.
- [26] B. Liang, Z. Chen, M. Guo, Y. Wang, Y. Wang, Space robot target intelligent capture system based on Deep reinforcement learning model, *J. Phys. Conf. Ser.* 1848 (1) (2021) 012078, <https://doi.org/10.1088/1742-6596/1848/1/012078>.
- [27] H. Li, D. Gong, J. Yu, An obstacles avoidance method for serial manipulator based on reinforcement learning and artificial potential field, *Int. J. Intell. Robot. Appl. (IJIRA)* 5 (2) (2021) 186–202, <https://doi.org/10.1007/s41315-021-00172-5>.
- [28] Y. Li, D. Li, W. Zhu, J. Sun, X. Zhang, S. Li, Constrained path planning of 7-DOF space manipulator via Deep reinforcement learning combined with artificial potential field, *J. Aerosp.* 9 (3) (2022), <https://doi.org/10.3390/aerospace9030163>. Article 163.
- [29] M. Jepma, S. Nieuwenhuis, Pupil diameter predicts changes in the exploration–exploitation trade-off: evidence for the adaptive gain theory, *J. Cogn. Neurosci.* 23 (7) (2011) 1587–1596, <https://doi.org/10.1162/jocn.2010.21548>.
- [30] M. Wilde, S.K. Choon, A. Grompone, M. Romano, Equations of motion of free-floating spacecraft-manipulator systems: an engineer’s tutorial, *Front. Robot. AI* 5 (2018) 41, <https://doi.org/10.3389/frobt.2018.00041>.
- [31] A. Zavoli, L. Federici, Reinforcement learning for robot trajectory design of interplanetary missions, *J. Guid. Control. Dyn.* 44 (8) (2021) 1440–1453, <https://doi.org/10.2514/1.G005794>.
- [32] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, M. Riedmiller, Deterministic policy gradient algorithms, 31st Int. Conf. Mach. Learn. (ICML) (2014) 387–395. Beijing, China, <https://proceedings.mlr.press/v32/silver14.html>.
- [33] T. Degris, M. White, R.S. Sutton, Off-policy actor-critic, *Int. Conf. Mach. Learn. (ICML)* (June 2012), <https://doi.org/10.48550/arXiv.1205.4839>. Edinburgh, United Kingdom.
- [34] C. Watkins, P. Dayan, Q-learning, *Mach. Learn.* 8 (1992) 279–292, <https://doi.org/10.1007/BF00992698>.
- [35] R. Bellman, Dynamic programming, *Science* 153 (1966) 34–37, <https://doi.org/10.1126/science.153.3731.34>.
- [36] T.P. Lillicrap, J.J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, D. Wierstra, Continuous control with Deep reinforcement learning, *ArXiv:1509.02971[cs.LG]* (September 9, 2015), <https://doi.org/10.48550/arXiv.1509.02971>.
- [37] R. Campa, K. Camarillo, M. Ceccarelli, in: *Robot manipulators*, M. Ceccarelli (Eds.), Unit Quaternions: A Mathematical Tool for Modeling, Path Planning and Control of Robot Manipulators, Teh, 2008, pp. 21–48, <https://doi.org/10.5772/6197>.
- [38] X. Yangsheng, T. Kanade (Eds.), *Space Robotics: Dynamics and Control*, vol. 188, Springer Science & Business Media, 1992.
- [39] C. Chen, H. Tang, J. Hao, W. Liu, Z. Meng, Addressing action oscillations through learning policy inertia, *Proc. AAAI Conf. Artif. Intell.* 35 (8) (2021) 7020–7027, <https://doi.org/10.1609/aaai.v35i8.16864>.